

贵州大学
GUIZHOU UNIVERSITY

实践（实验）报告

课程名称： Java Web 后端开发技术

学　　院： 示范性软件工程学院

授课教师： 丁玺润

专业班级： 软件工程 2403

学　　号： 2400770225

姓　　名： 皮宸西

小组成员： 皮宸西

2026 年 9 月 11 日

实验名称：	实验一：Spring Boot 项目搭建与 Maven 管理

一、实验目的：

1. 理解 Spring Boot 的定位、特性与 Spring 的关系，掌握其"自动配置、起步依赖"的核心思想。

2. 掌握使用 Spring Initializr 创建 Spring Boot 项目的完整流程，理解项目目录结构与各文件作用。

3. 掌握 Maven 依赖管理，理解 pom.xml 中 parent、起步依赖、插件的作用。

4. 掌握 application.yml 配置文件的基本写法，能够完成数据库、端口等基础配置。

二、实验环境：

硬件环境：PC 计算机（内存 8GB 及以上）。

软件环境：JDK 17、Maven 3.8+、IntelliJ IDEA 2023+、Spring Boot 3.x。

网络环境：可访问 Maven 中央仓库（或已配置国内镜像）。

三、实验内容：

1. 使用 Spring Initializr 创建"校园实训任务管理系统（CTTS）"后端项目骨架。

2. 配置 pom.xml，添加 Spring Web、MyBatis-Plus、MySQL 驱动等起步依赖。

3. 编写 application.yml，配置服务端口、MySQL 数据源。

4. 编写启动类，运行并验证项目可正常启动。

四、实验步骤（包含运行结果截图）：

（1）创建 CTTS 项目骨架

使用 Spring Initializr 生成"校园实训任务管理系统（CTTS）"的后端工程骨架：访问 https://start.spring.io ，按下述参数选择后生成并下载压缩包，解压到工作目录。

生成参数：Project = Maven、Language = Java、Spring Boot = 3.3.5、Group = com.gzu、Artifact = ctts、Name = ctts、Packaging = Jar、Java = 17。

生成的工程坐标为 com.gzu:ctts:0.0.1-SNAPSHOT，根包为 com.gzu.ctts，目录结构如图 1.1 所示。



图 1.1 项目目录结构

按 Maven 约定，src/main/java 存放 Java 源码，src/main/resources 存放配置文件与静态资源，src/test/java 存放测试代码；target 目录由 Maven 构建时自动生成，无需手工创建。

（2）配置 pom.xml，引入所需依赖

pom.xml 是 Maven 的工程描述文件。工程首先通过 parent 节点继承 spring-boot-starter-parent 3.3.5，从而直接获得依赖版本仲裁、源码编码、编译与打包插件的默认配置，无需逐一书写版本号；随后在 dependencies 节点中按功能引入起步依赖（starter），由 Spring Boot 自动带入该功能所需的全部传递依赖。关键配置如图 1.2 所示，各依赖的作用见表 1.1。

CTTS - pom.xml 依赖配置

pom.xml（节选：第 20 ~ 49 行）

```
20 |     <description>校园实训任务管理系统（CTTS）后端工程</description>
21 |
22 |     <properties>
23 |         <java.version>17</java.version>
24 |         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
25 |         <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
26 |         <!-- MyBatis-Plus 需显式指定版本（不在 Spring Boot 父工程的依赖管理中） -->
27 |         <mybatis-plus.version>3.5.9</mybatis-plus.version>
28 |     </properties>
29 |
30 |     <dependencies>
31 |         <!-- ① Web 场景启动器：内嵌 Tomcat + Spring MVC + Jackson -->
32 |         <dependency>
33 |             <groupId>org.springframework.boot</groupId>
34 |             <artifactId>spring-boot-starter-web</artifactId>
35 |         </dependency>
36 |
37 |         <!-- ② 持久层：MyBatis-Plus（Spring Boot 3 专用 starter） -->
38 |         <dependency>
39 |             <groupId>com.baomidou</groupId>
40 |             <artifactId>mybatis-plus-spring-boot3-starter</artifactId>
41 |             <version>${mybatis-plus.version}</version>
42 |         </dependency>
43 |
44 |         <!-- ③ MySQL 数据库驱动（版本由 Spring Boot 父工程统一管理） -->
45 |         <dependency>
46 |             <groupId>com.mysql</groupId>
47 |             <artifactId>mysql-connector-j</artifactId>
48 |             <scope>runtime</scope>
49 |         </dependency>
```

图 1.2 pom.xml 依赖配置节选

表 1.1 pom.xml 依赖清单及作用

依赖（artifactId）	**版本来源**	**作用**
spring-boot-starter-parent	3.3.5	父工程，统一仲裁依赖与插件版本
spring-boot-starter-web	继承父工程	内嵌 Tomcat + Spring MVC + Jackson，提供 Web 能力
mybatis-plus-spring-boot3-starter	3.5.9（显式指定）	持久层框架，简化 MyBatis 的单表增删改查
mysql-connector-j	继承父工程	MySQL 数据库 JDBC 驱动
spring-boot-starter-validatio	继承父工程	参数校验（JSR-380）

n		
spring-boot-devtools	继承父工程	开发期热部署，改完代码保存即自动重启
lombok	继承父工程	注解简化样板代码（@Data、@Slf4j 等）
spring-boot-starter-test	继承父工程	JUnit 5 + Spring Test + Mockito 测试支持

需注意：MyBatis-Plus 不属于 Spring Boot 官方依赖管理范围，必须在 properties 中显式声明 mybatis-plus.version 并在依赖中引用，否则会因找不到版本号而构建失败。

（3）编写 application.yml

在 src/main/resources 下新建 application.yml，依次配置服务端口、MySQL 数据源和 MyBatis-Plus。配置内容如图 1.3 所示，关键配置项的含义见表 1.2。

CTTS – application.yml

src/main/resources/application.yml

```yaml
server:
  # 服务端口
  port: 8080
  # 内嵌 Tomcat 的 URI 编码
  tomcat:
    uri-encoding: UTF-8

spring:
  application:
    name: ctts
  # 数据源：MySQL
  datasource:
    driver-class-name: com.mysql.cj.jdbc.Driver
    url: jdbc:mysql://localhost:3306/ctts?useUnicode=true&characterEncoding=utf8&useSSL=false&serverTimezone=Asia/Shanghai&allowPublicKeyRetrieval=true
    username: root
    password: 123456
    # HikariCP 连接池
    hikari:
      pool-name: CttsHikariPool
      minimum-idle: 5
      maximum-pool-size: 20
      connection-timeout: 30000
      idle-timeout: 600000
      max-lifetime: 1800000

# MyBatis-Plus 配置
mybatis-plus:
  configuration:
    # 开启驼峰命名自动映射：数据库 user_name -> 实体 userName
    map-underscore-to-camel-case: true
    # 开发期把 SQL 打印到控制台，便于调试
    log-impl: org.apache.ibatis.logging.stdout.StdOutImpl
  global-config:
    banner: false
    db-config:
      # 主键策略：数据库自增
      id-type: auto

# 应用元信息
info:
  app:
    name: 校园实训任务管理系统（CTTS）
    version: 0.0.1-SNAPSHOT
```

图 1.3 application.yml 配置内容

表 1.2 application.yml 关键配置项

配置项	取值	说明
server.port	8080	内嵌 Tomcat 的监听端口
spring.datasource.driver-class-name	com.mysql.cj.jdbc.Driver	MySQL 8 的 JDBC 驱动类
spring.datasource.url	jdbc:mysql://localhost:3306/ctts	目标数据库连接串，指向本机 ctts 库

spring.datasource.username / password	root / 123456	数据库账号与密码
spring.datasource.hikari 下的参数	minimum-idle=5 、maximum-pool-size=20 等	HikariCP 连接池参数
mybatis-plus.configuration.map-underscore-to-camel-case	true	开启驼峰映射：user_name → userName
mybatis-plus.global-config.db-config.id-type	auto	主键策略为数据库自增

工程同时提供了 sql/ctts.sql 建库脚本，用于创建 ctts 数据库以及 tb_task、tb_submission 两张业务表，便于后续实验直接接入数据访问层。

（4）编写启动类

在根包 com.gzu.ctts 下创建启动类 CttsApplication，源码如图 1.4 所示。@SpringBootApplication 是一个组合注解，等价于 @SpringBootConfiguration、@EnableAutoConfiguration 与 @ComponentScan 三者之和，因此启动类必须放在其它业务代码的最外层包，否则组件扫描不到。

CTTS – CttsApplication.java

src/main/java/com/gzu/ctts/CttsApplication.java

```java
package com.gzu.ctts;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
 * 校园实训任务管理系统（Campus Training Task System，CTTS）启动类。
 *
 * <p>{@code @SpringBootApplication} 是一个组合注解，等价于：
 * <ul>
 *   <li>{@code @SpringBootConfiguration}：声明为配置类；</li>
 *   <li>{@code @EnableAutoConfiguration}：开启 Spring Boot 自动配置（"约定优于配置"的核心）；</li>
 *   <li>{@code @ComponentScan}：以本类所在包 {@code com.gzu.ctts} 为根，扫描其下的
 *       {@code @Component}/{@code @Service}/{@code @Controller} 等组件。</li>
 * </ul>
 * 因此启动类必须放在其它业务代码的最外层包，否则组件扫描不到。
 */
@SpringBootApplication
public class CttsApplication {

    public static void main(String[] args) {
        SpringApplication.run(CttsApplication.class, args);
    }

}
```

图 1.4 启动类 CttsApplication.java 源码

（5）构建项目

在工程根目录执行 Maven 构建命令 `mvn -B clean package`。Maven 依次执行 clean、resources、compile、testResources、testCompile、test、jar、spring-boot:repackage 等阶段，最终生成可直接运行的可执行 jar（fat-jar）。构建结果如图 1.5 所示，输出 BUILD SUCCESS，且工程自带的单元测试 1 个全部通过。



图 1.5 Maven 构建结果（BUILD SUCCESS）

（6）启动项目并验证

执行启动命令 `java -jar target/ctts-0.0.1-SNAPSHOT.jar --server.port=8080`。启动日志如图 1.6 所示：内嵌 Tomcat 初始化并监听 8080 端口，Spring 上下文加载完成，最终输出 Started CttsApplication in 4.708 seconds，说明项目启动成功。

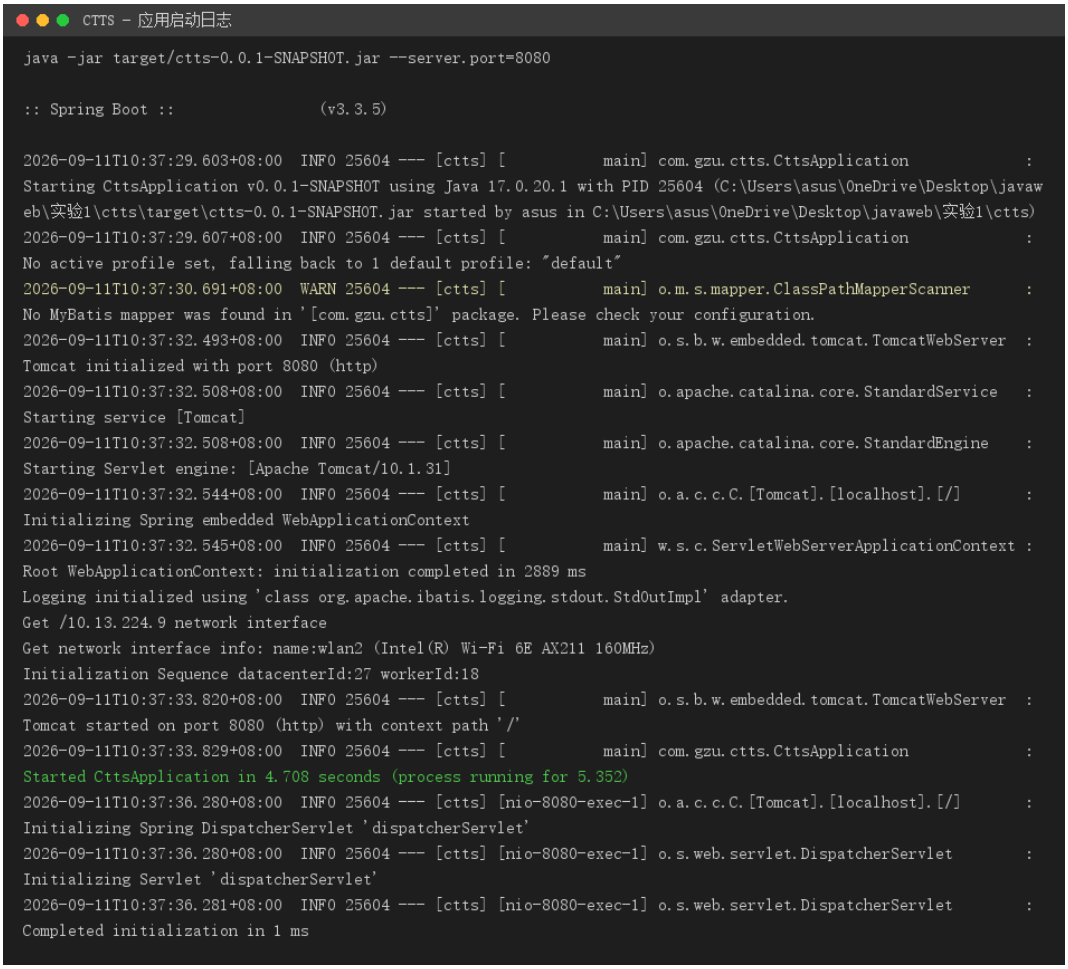


图 1.6 应用启动日志（内嵌 Tomcat 监听 8080 端口）

进一步验证结果如图 1.7 所示。8080 端口处于 LISTENING 状态；对 http://localhost:8080/ 发起请求，服务端返回 HTTP 404 及 Spring Boot 统一错误 JSON，同时在控制台打印 DispatcherServlet 初始化日志，说明请求确实被内嵌 Tomcat 接收并交由 Spring MVC 处理（本实验只搭建骨架、尚未编写 Controller，因此所有路径均返回 404 属预期行为）。此外以 --debug 模式启动时，自动配置条件评估报告中 DataSourceAutoConfiguration 、DataSourceConfiguration.Hikari 、MybatisPlusAutoConfiguration、ServletWebServerFactoryConfiguration.EmbeddedTomcat 等条目全部命中，说明数据源、持久层与 Web 容器的自动配置均已按 application.yml 正确生效。运行验证结果汇总见表 1.3。

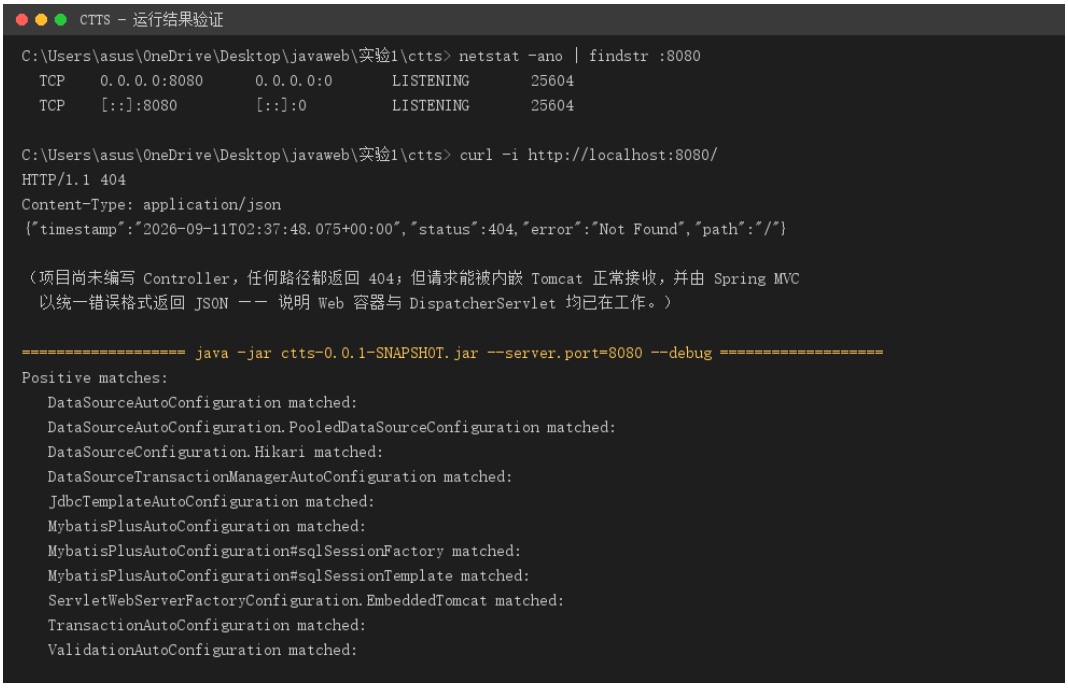


图 1.7 运行结果验证（端口监听、HTTP 响应与自动配置报告）

表 1.3 运行验证结果汇总

验证项	验证方式	预期结果	实测结果	结论
工程构建	mvn -B clean package	BUILD SUCCESS	BUILD SUCCESS，总耗时 35.055 s	通过
单元测试	随构建自动执行	全部通过	Tests run: 1, Failures: 0, Errors: 0	通过
端口监听	netstat -ano 过滤 8080	0.0.0.0:8080 LISTENING	0.0.0.0:8080 LISTENING（PID 25604）	通过
启动完成	应用控制台日志	输出 Started CttsApplication	Started CttsApplication in 4.708 seconds	通过
HTTP 响应	GET http://localhost:8	Web 容器可接收请求	HTTP 404 + 统一错误 JSON	通过

	080/			
自动配置	--debug 条件评估报告	数据源 / 持久层 / Web 均命中	相关 Positive matches 全部命中	通过

五、实验总结：

1. 对 Spring Boot 定位的理解。本实验用一个最小可运行的工程直观验证了 Spring Boot 的核心思想——"约定优于配置、自动配置"。整个工程没有编写一行 XML，仅一个 @SpringBootApplication 注解加上一份 application.yml，就完成了内嵌 Tomcat、Spring MVC、数据源（HikariCP）、事务管理器、持久层（MyBatis-Plus）的装配；用 --debug 打印的条件评估报告可以清楚看到这些自动配置类逐条命中。代价是"配置从哪来"被隐藏了，出问题时需要借助条件评估报告、属性绑定日志来定位，这正是后续实验要重点掌握的能力。

2. 对 Maven 依赖管理的理解。 父工程 spring-boot-starter-parent 通过依赖管理统一仲裁版本，子工程只写 groupId 与 artifactId 即可，从根本上避免了版本冲突；起步依赖则把"实现某项功能所需的全部依赖"打包成一个坐标。二者配合的逻辑是：依赖决定 classpath 上有什么，自动配置据此决定装配什么。实验中也遇到了例外——MyBatis-Plus 不在官方依赖管理范围内，必须自己声明版本，否则构建直接失败，这提醒我"继承父工程"并不等于"所有依赖都不用写版本"。

3. 对配置加载与优先级的理解。 实验中按 application.yml 配置了 server.port=8080，但首次启动时 Tomcat 却监听在另一个端口并报端口占用而启动失败。排查后确认是运行环境中存在同名环境变量，其优先级高于配置文件。Spring Boot 外部化配置的优先级为：命令行参数 > 环境变量 > 配置文件，因此改用 --server.port=8080 启动后一切正常。这个问题虽然由环境引入，却比单纯背结论更能让人记住配置优先级的实际影响。

4. 对"可验证"的认识。 本次复现的每一步都留下了可复查的证据：构建日志（BUILD SUCCESS、测试 1/1 通过）、启动日志（Tomcat started on port 8080、Started CttsApplication in 4.708 seconds）、端口监听状态、HTTP 响应，以及自动配置报告。这样一来"实验成功"就不是主观判断，而是别人照着重跑一遍也能得到一致结果的过程。

5. 不足与下一步。 由于本机尚未部署 MySQL 服务，application.yml 中的数据源目前只完成了配置层面的验证（HikariCP 与 MyBatis-Plus 的自动配置已命中且属性绑定正

确），还没有做真实的连库读写。后续实验将先在本机安装并启动 MySQL、执行 sql/ctts.sql 建库建表，再补齐实体类、Mapper 与 Controller，把任务发布、任务查询等接口跑通，用真实数据把持久层与 Web 层完整串起来。
六、教师评语：

评语教师：	丁奎润	实验评分：	